FORMATION REACT.JS · DÉMO

Démo 003-D · Le tableau d'objets

02-tableau-objets.js : les lignes ajoutées à chaque étape, et le résultat attendu.

Étape 1 · Tableau d'objets simple (id, name) (le départ)

```
// Tableau d'objets simple (id, name)
const users = [
  { id: 1, name: "Alice" },
  { id: 2, name: "Bob" },
  { id: 3, name: "Sara" },
];

console.log("Initial:", users);
```

Résultat attendu (console)

```
Initial: [
  { id: 1, name: 'Alice' },
  { id: 2, name: 'Bob' },
  { id: 3, name: 'Sara' }
]
```

Étape 2 · on ajoute : Trouver l'index par id (bon)

```
// IMPORTANT : Types + comparaison
// - name est une STRING (type primitif) -> comparaison par VALEUR (contenu)
//   Exemple : "Bob" === "Bob" -> true
//
// - {id: 2, name: "Bob"} est un OBJET -> comparaison par RÉFÉRENCE (adresse mémoire)
//   Exemple : {..} === {..} -> false (même contenu, mais objets différents)
//
// Donc :
// - indexOf marche très bien sur les primitives (string, number, boolean)
// - indexOf sur les objets marche seulement si c'est le MÊME objet (même référence)
// - pour chercher un objet par une propriété (id, name...), on utilise findIndex / find

// 1) Trouver l'index par id (bon)
// findIndex compare avec une condition (ici u.id === 2)
// Retour : l'index trouvé, sinon -1

const indexBob = users.findIndex((u) => u.id === 2);
console.log("Index de id=2 (findIndex):", indexBob); // 1
```

Résultat attendu (console)

```
Index de id=2 (findIndex): 1
```

Étape 3 · on ajoute : Trouver l'index par name (bon)

```
// 2) Trouver l'index par name (bon)
// Ici u.name est une string, donc comparaison par valeur

const indexSara = users.findIndex((u) => u.name === "Sara");
console.log("Index de Sara (findIndex):", indexSara); // 2
```

Résultat attendu (console)

```
Index de Sara (findIndex): 2
```

Étape 4 · on ajoute : Trouver l'objet (find) (bon)

```
// 3) Trouver l'objet (find) (bon)
// find retourne l'OBJET (pas l'index)
// Retour : l'objet trouvé, sinon undefined

const user1 = users.find((u) => u.id === 1);
console.log("User id=1 (find):", user1); // { id: 1, name: "Alice" }
```

Résultat attendu (console)

```
User id=1 (find): { id: 1, name: 'Alice' }
```

Étape 5 · on ajoute : Exemple "pas trouvé"

```
// 4) Exemple "pas trouvé"
// findIndex -> -1 si pas trouvé
// find -> undefined si pas trouvé

console.log(
  "Index id=99 (findIndex):",
  users.findIndex((u) => u.id === 99)
); // -1
console.log(
  "User id=99 (find):",
  users.find((u) => u.id === 99)
); // undefined
```

Résultat attendu (console)

```
Index id=99 (findIndex): -1
User id=99 (find): undefined
```

Étape 6 · on ajoute : PIÈGE : indexOf avec un NOUVEL objet ()

```
// 5) PIÈGE : indexOf avec un NOUVEL objet ()
// Ici on crée un nouvel objet { id: 2, name: "Bob" }
// Même contenu que users[1], mais pas la même référence mémoire
// Donc indexOf ne le trouve pas -> -1

console.log(
  "Piège indexOf (nouvel objet):",
  users.indexOf({ id: 2, name: "Bob" })
); // -1

// Petite preuve : deux objets identiques ne sont pas égaux
const a = { id: 2, name: "Bob" };
const b = { id: 2, name: "Bob" };
console.log("a === b ?", a === b); // false (objets différents en mémoire)
```

Résultat attendu (console)

```
Piège indexOf (nouvel objet): -1
a === b ? false
```

Étape 7 · on ajoute : indexOf marche si c'est la MÊME référence

```
// 6) indexOf marche si c'est la MÊME référence
// users[1] est l'objet Bob existant dans le tableau
// bobRef pointe vers EXACTEMENT le même objet
// Donc indexOf retrouve sa position -> 1

const bobRef = users[1];
console.log("Index bobRef (même référence):", users.indexOf(bobRef)); // 1

// Preuve : bobRef et users[1] sont le même objet
console.log("bobRef === users[1] ?", bobRef === users[1]); // true
```

Résultat attendu (console)

```
Index bobRef (même référence): 1
bobRef === users[1] ? true
```